

1. Introducción

El presente documento consolida el **Pipeline de Integración Continua (CI)**, el **Pipeline de Liberación**, el **Pipeline de Despliegue Continuo (CD)**, los **parámetros de configuración**, los **niveles de servicio**, las **métricas de monitoreo** y los **anexos técnicos** del sistema Ecoluz Urbana.

Se integra la información de las fases PHVA:

Fase H – Hacer (pruebas ejecutadas)

Fase V – Verificar (validación de conformidad)

Fase A – Actuar (acciones preventivas, correctivas y de mejora)

Así como los documentos base del sistema:

ERS / Requisitos del Sistema

Factibilidad técnica-operativa del proyecto

Arquitectura, metodologías y comparativas técnicas

2. Justificación del Pipeline de CI/CD

2.1 Razones estratégicas para implementar CI/CD

El sistema Ecoluz procesa datos de estadísticas energéticas, esto exige:

- **Disponibilidad ≥ 99.5 %**
- **Tiempos de respuesta del backend $< 400\text{--}500$ ms**

Ante estos requisitos, CI/CD permite:

- Detectar errores antes de llegar a producción.
- Validar cambios automáticamente en cada commit.
- Asegurar que *main* siempre esté en estado desplegable.
- Reducir tiempos de entrega y riesgos en despliegues.

2.2 Justificación del flujo seleccionado: GitHub Flow / GitLab Flow híbrido

El proyecto utiliza un modelo híbrido basado en *GitHub Flow* adaptado a entornos (Staging y Production) como propone GitLab Flow.

Las razones:

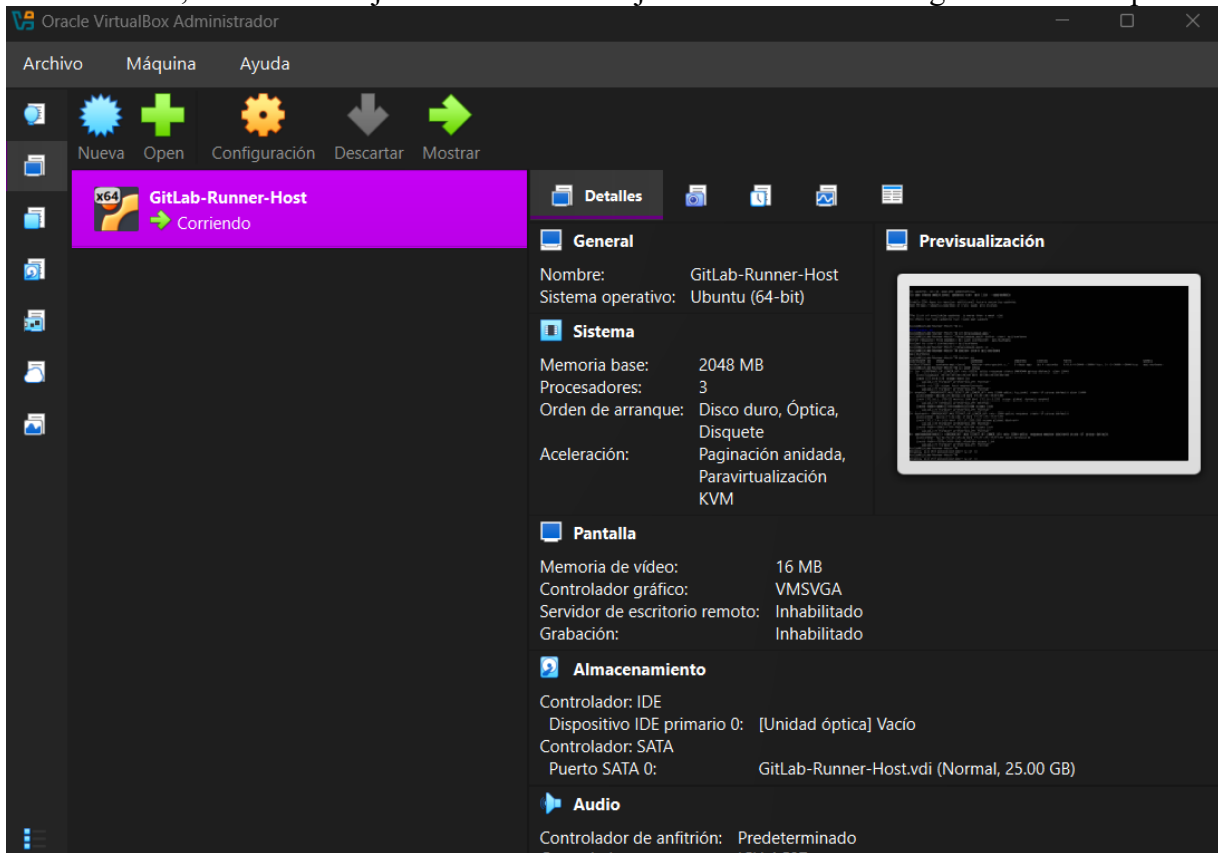
- **Simplicidad** (ramas pequeñas por tarea).
- **Despliegue continuo automático** al aprobar un MR/PR.
- **Revisión temprana** del código por el equipo.
- **Menor complejidad** que Git Flow tradicional.
- **Compatibilidad con Render y Vercel**, que despliegan automáticamente desde *main*.

Este flujo se alinea con:

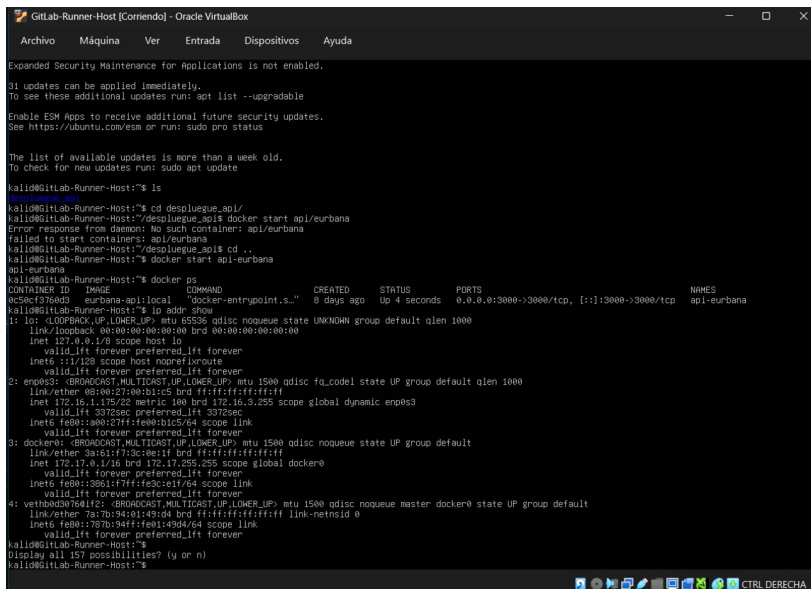
- Acciones correctivas de la Fase A (resolver endpoints incompletos, mejorar excepciones, prevenir errores futuros).

- Pruebas automatizadas destacadas en el documento de Integración Continua.

Además debido a la falta de infraestructura para realizar las distintas pruebas se opto por crear una VM en Linux, esto con el objetivo de tener un objetivo hacia el cual dirigir las distintas prueba



optamos también por una mejor seguridad y sobre todo organización usar contenedores en Docker, levantando la api por medio de estos



Arquitectura del Pipeline de Integración Continua (CI)

Cada *push* o *merge request* hacia la rama **main** activa el pipeline, compuesto por 7 etapas principales.

3.1 Etapa 1 – Build automático

Valida que el backend (Node.js + Express) y el frontend (React) compilen correctamente. Se basa en configuraciones documentadas:

- Split de código, lazy loading del frontend.
- Pool de conexiones en backend.

Errores de dependencias o compilación detienen la ejecución.

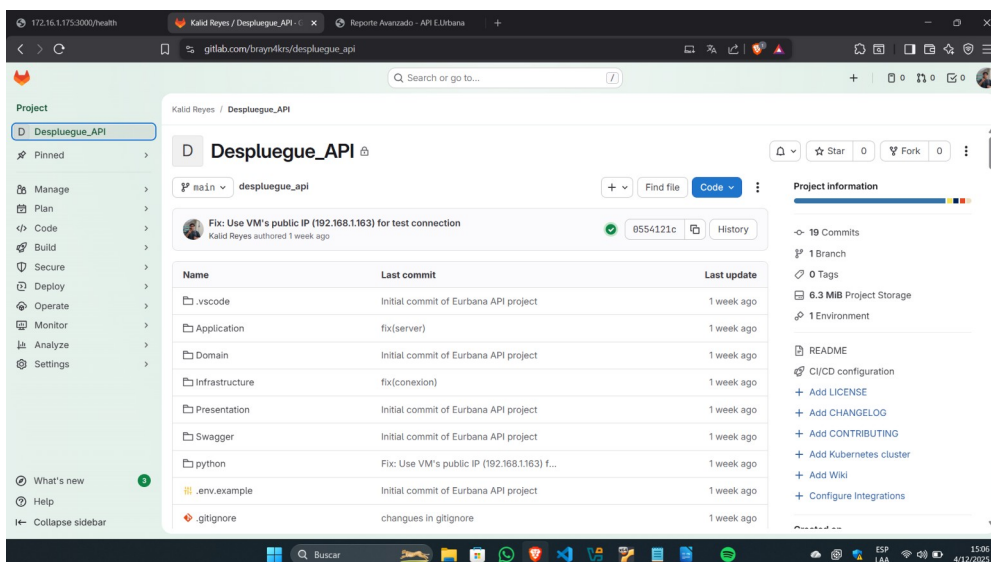
3.2 Etapa 2 – Análisis de código estático

Herramientas: ESLint, Pylint, Objetivo: detectar vulnerabilidades y malas prácticas.

Basada en lo descrito en:

- Integración Continua y Pruebas en E. Urbana.

3.3 Etapa 3 – Pruebas unitarias



Runner activo para hacer las pruebas en el servidor

Tecnologías:

- **Jest** para React.
- **Mocha / JUnit** para lógica de negocio.

Validan componentes aislados y módulos críticos.

3.4 Etapa 4 – Pruebas de integración

Pruebas entre:

- API ↔ MongoDB
- Frontend ↔ API
- Sensores simulados ↔ API

Basado en pruebas definidas en Fase H.

3.5 Etapa 5 – Pruebas de aceptación (E2E)

- Flujos completos: Login, mapa de luminarias, consulta de estadísticas, filtros, etc.
- Validación de roles: admin, operador, técnico.

3.6 Etapa 6 – Pruebas de rendimiento

Con JMeter:

- 100 usuarios simultáneos mínimo.
- Carga masiva de registros IoT.

Referenciado en documentación de métricas.

3.7 Etapa 7 – Generación de reportes

El pipeline produce:

- Reporte HTML del estado del sistema (40 métricas evaluadas).
- Logs del análisis estático.
- Capturas de errores y outputs de Cypress.

Reporte Avanzado de Métricas - API E.Urbana	
Generado el 2025-11-27 01:56:12	
◆ Métricas Globales	
Métrica	Valor
Tiempo total (s)	0.27
Promedio global de respuesta (ms)	5000
Latencia mínima global (ms)	5000
Latencia máxima global (ms)	5000
Desviación global (ms)	0.0
Disponibilidad promedio (%)	0.0
Errores totales	99
Throughput global (req/s)	362.65
Payload promedio global (bytes)	0
Uso de CPU (%)	0
Uso de Memoria (%)	0
P99 latencia (ms)	5000
Estabilidad del servicio (%)	100.0
Endpoints exitosos (%)	0.0
Tasa de fallos (%)	100.0
Pérdida de paquetes (%)	100.0

Archivo HTML correspondiente a las pruebas hechas

Basado en:

- Script automatizado de métricas.

4. Pipeline de Liberación (Release Pipeline)

Project

D Despluegue_API

Pinned

Manage

Plan

Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure

Deploy

Operate

Monitor

Analyze

Settings

What's new

Help

Collapse sidebar

Kalid Reyes / Despluegue_API / Pipelines

All 19

Finished

Branches

Tags

View analytics

Clear runner caches

New pipeline

Filter pipelines

Search

Show Pipeline ID

Status	Pipeline	Created by	Stages	Actions
Failed 00:00:14 1 week ago	delete dotenv #2181458541 main c4ed1a9b branch			Refresh Download
Failed 00:00:22 1 week ago	deploy in yml #2181142017 main 95d034e6 branch			Refresh Download
Canceled 00:01:12 1 week ago	changues in gitignore #2179652502 main d5d64e6a branch			Refresh Download
Failed 00:29:45 1 week ago	Configure CI/CD pipeline and update pr... #2179578904 main f2367bdd branch			Refresh Download

<

1

2

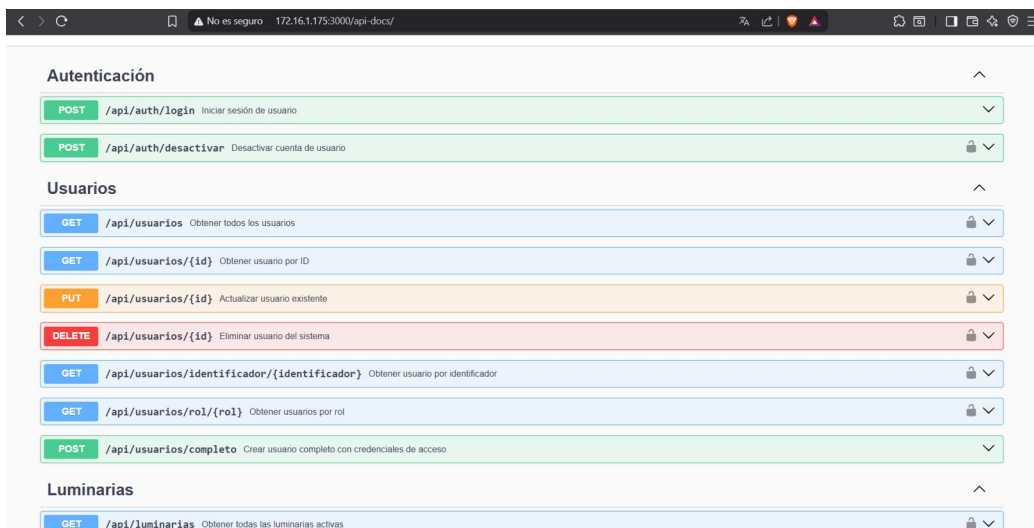
>

Pruebas reales con Pipelines

El pipeline de liberación verifica que una versión pueda salir de Staging hacia Producción.

Validaciones antes de liberar

1. **Pruebas manuales avanzadas** (Mapa, gráficos, JWT, Smartwatch).
Basado en pruebas manuales descritas en parámetros de herramientas.
2. **Validación contra requisitos funcionales y no funcionales:**
 - RF-05, RF-06, RF-10, RF-11, RF-12.
 - RNF-01, RNF-05, RNF-09, RNF-14.
3. **Verificación del modelo de datos**
 - Usuario, Luminaria, Consumo.
4. **Confirmación del cumplimiento de niveles de servicio (SLA):**
 - < 400–500 ms por endpoint.
 - Disponibilidad ≥ 99 %.
 - Tasa de errores <2 %.
5. **Revisión del estado de la base de datos**
 - Índices activos.
 - Limpieza de registros antiguos.
(Pruebas incluidas en Fase H).



Sólo si todo se cumple, se permite el despliegue manual a Producción.

5. Pipeline de Despliegue Continuo (CD)

5.1 Flujo Staging → Production

1. **Push/Merge a main**
Despliega automáticamente en Staging:
 - Backend en Render.
 - Frontend en Vercel.
2. **Pruebas manuales finales (Smoke Tests)**
 - Login

- Carga del mapa
 - Estadísticas
 - Verificación de endpoints críticos
3. **Activación manual del job “Deploy to Production”**
- Evita despliegues defectuosos.
 - Permite control académico durante revisiones.

5.2 Entornos requeridos

Entorno	Propósito	Hospedaje
Local	Desarrollo, pruebas iniciales	Máquina local
Staging	Validación preproducción	Render / Vercel
Producción	Versión final	Render / Vercel

Ambos entornos están configurados mediante variables de entorno (.env).

6. Parámetros de Configuración de Herramientas

Integrado de la documentación de parámetros y pruebas.

6.1 Backend (Node.js + Express)

- JWT_SECRET
- Tiempos de expiración
- Pool de conexiones MongoDB
- Manejo de concurrencia para IoT
- Logs, middleware de errores

6.2 Base de datos (MongoDB Atlas)

- Índices: fecha, zona, ID luminaria
- Backups semanales
- Límite de conexiones activas

6.3 Frontend (React)

- VITE_API_URL
- REFRESH_INTERVAL
- Code splitting y lazy loading

6.4 Infraestructura

Render / Vercel variables:

- NODE_ENV
- ENABLE_LOGS

7. Niveles de Servicio (SLAs) y Métricas

Basado en el documento de monitoreo /health.

7.1 Rendimiento

- Tiempo de respuesta: ≤ 400 ms
- Latencia promedio: ≤ 200 ms
- Disponibilidad: ≥ 99.5 %
- Memoria usada: ≤ 300 MB

7.2 Estabilidad

- Errores 5xx diarios: ≤ 3
- Conexiones activas DB: ≤ 50

7.3 Seguridad

- Expiración JWT: 24 h
- Intentos fallidos por usuario: ≤ 5

7.4 Infraestructura

- Tiempo promedio de despliegue: ≤ 2 min
- SSL activo

8. Monitoreo del Sistema

El endpoint **/health** expone más de 25 métricas:

- Estado del sistema
- Uptime
- Memoria
- CPU
- Latencia promedio
- Tasa de errores
- Depuración del event loop
- Estado de dependencias

Basado en:

Logs en consola de Render

- Alertas recomendadas para futuro: Logtail, Datadog

9. Acciones Correctivas, Preventivas y de Mejora (Resumen PHVA)

Integrado desde el documento de Actuación.

Acciones correctivas

- Completar endpoints faltantes.
- Mejorar manejo de errores 500.

Acciones preventivas

- Integrar pruebas automatizadas en CI.
- Creación de datos semilla uniformes.

Acciones de mejora

- Optimizar consultas con índices.
- Refinar UI del dashboard.
- Consolidar documentación en un repositorio.

11. Gestión de Incidencias

La gestión de incidencias permite registrar, clasificar, analizar y resolver cualquier problema detectado durante el desarrollo, pruebas, liberación, despliegue y operación del sistema Ecoluz Urbana. Su propósito es garantizar la trazabilidad de los errores, favorecer la mejora continua y reducir la probabilidad de fallas en producción.

11.1 Objetivos de la gestión de incidencias

- Detectar y documentar de forma uniforme cada error o comportamiento inesperado.
- Priorizar incidencias según impacto técnico y operativo.
- Asociar cada incidencia a commits, pruebas automatizadas o despliegues específicos.
- Mantener historial para auditorías y retroalimentación del ciclo PHVA.
- Evitar reincidencia mediante análisis de causas raíz.

11.2 Flujo de trabajo para incidencias

1. Detección

Ocurre durante pruebas unitarias, integración, aceptación, rendimiento o en ambiente productivo.

Ejemplos típicos documentados en fases H y V:

- Endpoints incompletos.
- Errores 500 durante operaciones CRUD.

2. Registro

El equipo captura la incidencia en GitHub Issues o Notion.

Datos mínimos:

- Descripción del problema
- Pasos para reproducir
- Evidencia (capturas, logs, payloads)
- Severidad
- Componente afectado (API, BD, Frontend, IoT)

3. Clasificación y priorización

- **Crítica:** afecta disponibilidad, seguridad o integridad de datos.

- **Alta:** impide operaciones principales (login, consultar mapa).
 - **Media:** afecta funciones secundarias.
 - **Baja:** detalles visuales o errores no obstructivos.
4. **Asignación**
La incidencia se asigna al responsable del módulo afectado.
Esto coincide con lo indicado en la fase A, donde se establecen responsables.
 5. **Resolución**
El programador genera un commit asociado y una rama (fix/...) con solución.
 6. **Validación**
El pipeline ejecuta nuevamente pruebas (unitarias, integración y aceptación).
 7. **Cierre y documentación**
Se registra el commit de solución, release asociado y se actualiza historial.

11.3 Tipos de incidencias comunes detectadas en el proyecto

- **Errores 500** por fallas en excepciones.
- **Endpoints faltantes** (luminarias CRUD).
- **Datos no persistidos** (consumo masivo).
- **JWT inválidos o expirados.**
- **Errores de validación 400** correctamente controlados.
- **Lentitud en consultas masivas** (resuelto con índices propuestos).

11.4 Análisis de causa raíz

Métodos usados:

- Revisión de logs de Render.
- Trazas en endpoints probados con Postman.
- Comparación con requisitos funcionales y no funcionales.

11.5 Integración con CI/CD

Cada incidencia cerrada requiere:

- Prueba automatizada añadida o actualizada.
- Commit etiquetado (fix #ID).
- Seguimiento del pipeline para verificar no regresiones.

12. Versionamiento del Sistema

El versionamiento asegura el control histórico del código, documentación, despliegues y artefactos generados. Para Ecoluz Urbana se utiliza un enfoque combinado entre *SemVer* (versionado semántico) y *ramas controladas* (GitHub Flow con ambientes).

Este esquema garantiza trazabilidad, estabilidad y capacidad de revertir versiones rápidamente.

12.1 Estrategia de versionamiento SemVer

Formato:

MAJOR.MINOR.PATCH

- **MAJOR** – Cambios incompatibles o refactorizaciones profundas.
- **MINOR** – Nuevas funcionalidades compatibles.
- **PATCH** – Corrección de errores, parches pequeños, hotfixes.

Ejemplos para el proyecto:

- 1.0.0 – Primera versión funcional del backend (API estable).
- 1.1.0 – Inclusión de estadísticas avanzadas y endpoints nuevos.
- 1.1.3 – Corrección de manejo de errores 500 en luminarias.
- 2.0.0 – Integración del smartwatch o rediseño global.

12.2 Organización de ramas en GitHub

Basada en flujo documentado en CI.

main

Producción. Siempre estable.

Automáticamente desplegada en Render/Vercel.

dev

Integración de funcionalidades.

Aquí se ejecuta el pipeline completo antes de liberar.

feature/

Para nuevas funciones:

feature/mapa, feature/auth, feature/smartwatch...

fix/

Para incidencias críticas:

fix/error500, fix/token-expired...

release/

Preparaciones de nuevas versiones:

release/1.1.0

Este modelo facilita el CI/CD automático y controlado.

12.3 Tags y Releases

Cada versión estable recibe un **tag**:

v1.0.0, v1.0.1, v1.1.0, etc.

Los releases contienen:

- Binarios o builds (si aplica).
- Notas de cambios (changelog).
- Lista de incidencias cerradas.

- Módulos afectados.

12.4 Changelog estructurado

Formato recomendado basado en Conventional Commits:

- **feat:** nueva funcionalidad
- **fix:** corrección
- **docs:** cambios documentales
- **refactor:** cambios sin afectar comportamiento
- **perf:** mejoras de rendimiento
- **test:** actualización de pruebas

Ejemplo de changelog:

```
## v1.1.3 - 2025-11-20
fix: manejo de errores 500 en luminarias
fix: validación de JWT expirado
perf: agregado índice en colección de consumo
docs: actualización del modelo de sistema
```